UID:

1) **Algorithm:**

1. Initialize an array of size n, where in-deg[v] = in-degree of v in the remaining graph.
2. Initialize a queue Q and add all nodes with in-degree 0.
3. Initialize a list, L to output the ordering.
4. While (Q is not empty)
    a. Pop a node v off of the queue and append to L.
    b. For each edge (v,w) in G that is incident to v:
        i. Decrement in-deg[w] by 1
        ii. If in-deg[w] == 0
            1. Add w to Q.
5. If the length of L is equal to the total number of nodes - G is a DAG, L is a valid topological order
    a. Output L
6. If the length of L is less than total number of nodes
    a. A certain node(s) was not processed because there were no nodes left with zero in-degrees (which is how they're picked), meaning a cycle exists.
    b. Initialize an empty list, C.
    c. For each node i
        i. If in_deg[i] > 0
            1. Append i to C
    d. Output C, G is not a DAG.

Proof:

Correctness: Assume for the sake of contradiction that the algorithm outputs a topological ordering when a cycle exists. If G contains a cycle, that means there are some nodes left with an in-degree > 0. They would never have been added to Q and never appended to L, so L would not be the length of the number of nodes. Since L < n, C would be outputted, not an ordering. If L was outputted, this is a contradiction since only L OR C is output.

Assume for the sake of contradiction that the algorithm outputs a cycle when a topological ordering exists. If an ordering exists, the graph must be a DAG and have no cycles. Thus, all nodes must eventually have in_deg == 0 and be added to Q and eventually L; this results in L having all the nodes, and thus L would be outputted. If a cycle was outputted instead, this is a contradiction since only C or L is output.

Termination: Each node will only be processed at most once, so the while loop eventually terminates. Once the while loop terminates, we either output L or C, ending the algorithm. Overall, there are a finite number of nodes so the for and while loops will terminate.

Completeness: We successfully either output a list with a topological ordering, if one can be produced, or a cycle, if no ordering exists because a cycle does.

Time Complexity Analysis:

1. Initialization of an array with degrees is O(m+n), mentioned in class.
2. Initialization of queue that can at most be n nodes: O(n)
3. Initializing a list: O(1)
4. While loop overall: Each edge incident to a node v is processed. This is O(out-deg(v)), as discussed in class. At most, it can be 2m.
5. Outputting L: O(1)
6. Outputting Cycle: O(n)
7. Overall: The dominating complexity is O(m+n).

2) Algorithm:

1. Run BFS, starting from S.
2. Initialize Discovered as an array size n with all values set to False
3. Initialize LayerCount as an array of size n with all values set to 0.
4. Initialize Q as a queue
5. Initialize variable L to keep track of the current layer, start at 0.
6. Set LayerCount[0] = 1 (the starting node only)
7. Add s to Q
8. Set Discovered[s] = True
9. While (Q is nonempty):
    a. Pop a node u from Q
    b. For every neighbor v of u:
        i. if Discovered[v]==False:
            1. Set Discovered[v] = True
            2. Push v to Q
            3. LayerCount[L] = LayerCount[L] + 1
    c. L++
10. For all elements in Layers 1 and beyond
    a. If LayerCount[i] == 1
        i. Return i

Proof:
Correctness: Assume, for the sake of contradiction the path length is greater than n/2 and we had 2 nodes in each layer. This means that path to t has n nodes in it (n/2 * 2) – and this is a contradiction because t could not be a node in the graph then. We only have n total nodes. Therefore, if you find the layer with only one node, i, in it, you can disconnect the path because there is no other option for i's parent to go to to reach t. The parent has no other children, and thus cannot make it to t.

Termination: The algorithm only allows each n node to be added to Q once. When it is Discovered, it is not added again. Therefore, we know the while loop will end. The algorithm also ensures that false or true is returned, ending the algorithm.

Completeness: Since we use BFS, we accurately make sure to check every single possible path and find a layer with only one node in it. We are also keeping track of every single layer, and we know we are visiting every layer because of the nature of BFS.

Time Complexity Analysis: I used the BFS algorithm which has time O(m+n). My modification is initializing a new array, O(n), and variable O(1). In the loop, I only added an incrementing step, adding O(1). 9.c. also takes O(1). The last for loop takes at most O(n) time if we had a node in each layer. Overall, the dominant term is still O(m+n), so this is the time complexity overall.

3) Algorithm:

Connected graph is already formed, where the edges represent the judgements.
Discovered array to mark what is visited.
Labels array to make the species A or B.

1. Initialize Discovered as an array of size n with all values set to false.
2. Initialize Labels as an array of size n with all values set to None.
3. Initialize Q as a queue.
4. Add i, any starting butterfly to Q.
5. Set Discovered[i] = true.
6. Assign label 'A' to i.
7. While (Q is not empty)
    a. Pop a node u from Q
        i. For every neighbor j of u in u's "same" linked list
            1. If Discovered[j] = false
                a. Set Discovered[j] = true.
                b. Add j to Q.
                c. Assign it the same label as u.
            2. If Discovered[j] = true and its Label is different than u
                a. Judgements are not consistent - return false.
        ii. For every neighbor j of u in u's "different" linked list
            1. If Discovered[j] = false
                a. Set Discovered[j] = true.
                b. Add j to Q.
                c. Assign it the opposite label as u.
            2. If Discovered[j] = true and its Label is same than u,
                a. Judgements are not consistent - return false.
8. Return true

Proof:
Correctness: I used the BFS algorithm from class, with some modification. Specifically, I added steps i and ii. Assume, for the sake of contradiction, that the algorithm returns true when the judgements are actually not consistent. However, when butterfly 1 is in butterfly 2's same list

and has the opposite label, we would return false. And if butterfly 3 is in butterfly 2's opposite list but has the same label, we would immediately return false. Thus, it is a contradiction and impossible that we return true and the judgements are invalid.

Terminates: The algorithm only allows each n butterfly to be added to Q once. When it is Discovered, it is not added again. Therefore, we know the while loop will end. The algorithm also ensures that false or true is returned, ending the algorithm.

Completeness: We check all possible relationships, since we iterate through each butterfly's existing same or different judgements. We only return true if ALL the judgements are correct and consistent with one another, or possible.

Time Complexity Analysis:

1. Initializing Discovered: O(n)
2. Initializing Labels: O(n)
3. Initializing Q: O(1)
4. Steps 4-6: O(1)
5. In each iteration of the while loop, it takes the deg(node i) time. In total for n nodes, we have at most 2m degrees or edges we are processing (the sum of the length of both linked lists) – meaning this contributes O(2m) time overall, simplifying to O(m)
6. Total: O(2n) + O(m) → O(n+m)

4) Algorithm:

Let s be the starting node.

1. Initialize Discovered as an n-size array with all values set to false.
2. Initialize Distance (stores length of shortest path to this node) as an n-size array of ints with all values set to infinity, except Distance[s] = 0.
3. Initialize Paths (stores number of shortest paths to this node) as an n-size array of ints with all values set to 0, except Paths[s] = 1.
4. Initialize Q as a queue.
5. Add s, the starting node, to Q.
6. Set Discovered[s] = true.
7. While (Q is not empty)
    a. Pop a node u from Q
        i. For every neighbor v of u
            1. If Discovered[v] = false
                a. Set Discovered[v] = true.
                b. Add v to Q.
                c. If Distance[v] > Distance[u] + 1
                    i. Distance[v] = Distance[u] + 1
                    ii. Paths[v] = Paths[u]
                d. If Distance[v] = Distance[u] + 1
                    i. Paths[v] = Paths[v] + Paths[u]

8. Return Paths[w]

Proof:

Completeness: The BFS approach ensures every node from the current once is visited, making sure that we examine every single possible path. We also calculate a shortest path from the starting node to the destination at each step, ensuring that at any given moment we have the most accurate shortest path length and number of shortest paths. In case a path does not exist, the algorithm will accurately return 0 since we initialized Paths[w] as that. Finally, we output Paths[w], the number of shortest paths there are to the destination.

Termination: The algorithm only allows each n node to be added to the Q once. Once it is Discovered, it is not added again. The while loop goes till Q is empty, and since each node is added only once, the queue is empty once every node is visited. Therefore, we know the while loop will end. The algorithm returns the number of shortest paths to w at the end, ending the algorithm.

Correctness: This is an adaptation of BFS, which we proved correct in class. The only thing I added was the Distance and Paths arrays, and the steps involving those.

- Path Length: Suppose for the sake of contradiction, that a shorter path exists from s to w. BFS visits all shortest paths to every node, i, that is in between s and w. If we are
    - 1) iteratively finding the shortest path from s to the nodes before w
    - 2) updating the shortest paths of each node everytime a new neighbor is discovered
    - 3) saving only the SHORTEST path found to a node i

  then we have to have found the shortest path overall from s to w. The only way the earlier claim is true that a shorter path exists is if we did not explore all possible paths and nodes, which is a contradiction to BFS.

- Path Count: Suppose for the sake of contradiction, that the number of shortest paths is wrong. BFS visits all shortest paths to every node, i, that is in between s and w. If we are
    - 1) iteratively checking if the current path is less or equal to the shortest path from s to the nodes i before w
    - 2) updating the count of shortest lengths if 1) is true everytime a new neighbor is discovered
    - 3) saving the number of shortest paths found to every node i between s and w

  then we have to have accurately kept track of the number of shortest paths overall from s to w. The only way the earlier claim is true that we miscounted shortest paths is if we did not explore all possible paths and nodes, which is a contradiction to BFS.

Time Complexity:

- Initializing the arrays in steps 1-3 takes O(n) time
- Initializing Q is O(1)
- The rest of the steps before the while loop take time O(1)
- While loop: You visit each node, and then visit each neighbor of the node, which means each iteration is O(deg(node)). This adds up to the total degrees, which is 2m time, since there are m edges.
- Total: After simplifying, O(m+n).

5) Algorithm:

Since we want to signify time order, we can use a directed graph.
If fact 1 in the problem is true, let $d_i < b_j$
If fact 2 in the problem is true, let $d_i < d_j$
If fact 3 in the problem is true, let $b_j <= d_i$ or $b_i <= d_j$

1. For each person P_n, let b_n represent their birth date and d_n represent their death date.
2. Create a graph with two nodes for each person n, represented by b_n and d_n.
3. If fact 1 is true, add a directed edge (d_i, b_j).
4. If fact 2 is true, add a directed edge (d_i, d_j).
5. If fact 3 is true, add directed edges (b_j, d_i) and (b_i, d_j).
6. Apply topological sorting algorithm as shown below.
7. Initialize an array of size n, where in-deg[v] = in-degree of v in the remaining graph.
8. Initialize a queue Q and add all nodes with in-degree 0.
9. Initialize an empty list, L to output the ordering.
10. While (Q is not empty)
    a. Pop a node v off of Q and append to L.
    b. For each edge (v,w) in G that is incident to v:
        i. Decrement in-deg[w] by 1
        ii. If in-deg[w] == 0
            1. Add w to Q.
11. Output L.

If the empty list is returned, there is no topological ordering of the people. No such dates exist that make the facts collected consistent with a true timeline.

If not, a topological ordering exists, and you can assign birth and death dates based on that node order outputted in L. As long as each date is after the one preceding it, it is valid birth and death dates for all the facts to hold true. To get the proposed dates: Start the first birth node in L at Jan 1, 1980, and assign the next node Feb 1, 1980 and keep going till December is reached. Once the end of the year is reached, increment the year and repeat the month changing. The starting year can be adjusted based on how many nodes there are, if you need more or less.

Proof:
Completeness: We build an edge for every possible fact, so we are checking for completeness of consistency with the timeline. If the ordering exists, the sort algorithm will output one fulfilling the date requirement. If not, it is an empty list, meaning no such dates exist, satisfying the instructions. There are inconsistencies, or cycles, detected.

Termination: The while loop terminates when the queue is empty, and each node is added to the queue at most once. Each edge is then processed exactly once, when a node is dequeued. Therefore, the algorithm will terminate after all nodes and edges are processed.

Correctness: I used our topological sort algorithm from class. As proved in class, if G is a DAG, then Topological Sort(G) outputs a topological ordering on G. Assume, for the sake of contradiction, that a cycle exists and a valid topological ordering is outputted in L. If a cycle exists, at least one node has non-zero in-degrees, and it would not be processed by the algorithm and there would not be a valid output in L. This would be a contradiction, therefore we know that G is a DAG and no cycles exist if something is outputted in L. This means we can do topological sort, and that output is the valid ordering of dates. If G is not a DAG and a cycle exists, we cannot have a topological ordering, thus L is empty.

The only addition I made was actually drawing the graph, which is just the earlier date pointing to the later date, based on the facts.

Time Complexity:

1. Drawing the nodes: Since there's n people and 2 nodes each: O(2n)
2. Iterate through the m facts in a list (assuming m) and add edges: O(m)
3. Topological sort:
    a. Initializing the array, as discussed in class: at most O(m+n)
    b. Initializing a queue with all nodes with in-deg 0: at most O(n)
    c. Initializing list: O(1)
    d. Overall in the while loop each edge incident to a node v is processed. This is O(out-deg(v)), as discussed in class. At most, it can be 2m.
4. Overall, summing these up the dominant term gives us overall time complexity O(m+n)